LAÇOS
FOR · WHILE · UNTIL

*O expurgo da madrugada rodava havia meses até a coleta gravar `relatorio de marco.log`. Naquela noite o `gzip` foi chamado três vezes — com relatorio, de e marco.log —, o log real continuou crescendo e um mês depois a partição encheu. O erro não estava no `for` nem no `gzip`: estava na **fonte da lista**.*

058
GUIAS LINUX

DE ONDE VEM A LISTA — O LAÇO ITERA PALAVRAS, NÃO ARQUIVOS

`for f in $(ls *.log)`	`relatorio` `de` `marco.log`	**3 iterações.** A saída virou **texto** e o `IFS` recortou nos espaços; um nome com `*` ainda se expandiria outra vez. Não existe caso em que iterar sobre `ls` seja melhor.
`for f in *.log`	`relatorio de marco.log`	**1 iteração.** O *globbing* entrega **itens**, não uma cadeia de caracteres: espaço e quebra de linha sobrevivem inteiros. Vale igual para `{1..10}` e `"${arr[@]}"`.
`while IFS= read -r -d '' f` `  < <(find … -print0)`	`relatorio de marco.log`	**1 iteração**, e também em subdiretório e com `\n` no nome. O separador é o **byte nulo** — o único caractere que o kernel proíbe num nome de arquivo.

Glob que não casa não vira lista vazia: em diretório sem `*.log`, o laço roda **uma vez** com `f` valendo o texto literal `*.log`. Conserte com `shopt -s nullglob` ou com `[[ -e $f ]] || continue` na primeira linha do corpo. E se o corpo é uma chamada única a um comando, dispense o laço: `find … -exec gzip {} +` ou `xargs -0`.

QUEM TESTA O QUÊ, E QUANDO

`for x in lista`

expande a lista
uma única vez

↓ sobrou item?

x = próximo
executa o corpo ↺

Acrescentar arquivos ao diretório no meio de um `for f in *` **não** muda o que já foi expandido.

`for (( ; ; ))`

i=0 · i<n · i++
contexto aritmético

↓ enquanto verdadeiro

executa o corpo ↺

A forma certa quando o limite é **variável**: `{1..$n}` não funciona — chaves expandem antes das variáveis.

`while comando`

o comando
saiu com **0**?

↓ sim

executa o corpo ↺

Reavalia **a cada volta**: serve para esperar estado externo — e erro na condição vira laço infinito, não mensagem.

`until comando`

o comando
saiu com **0**?

↓ **não**

executa o corpo ↺

É o `while !` escrito de forma legível: `until systemctl is-active --quiet pg; do sleep 2; done`.

O IDIOMA DE LEITURA — TRÊS DETALHES OBRIGATÓRIOS

while read — linha a linha, e por campos

```
# IFS= preserva espacos das pontas | -r preserva a \ | < depois do done evita o subshell
$ while IFS= read -r linha || [[ -n $linha ]]; do
>     printf '[%s]\n' "$linha"
> done < dados.txt          # o || pega a ultima linha sem \n final

# com valor, o IFS divide em campos; o resto cai na ultima variavel
$ while IFS=: read -r usuario _ uid _ _ _ shell; do
>   (( uid >= 1000 && uid < 65534 )) && printf '%-12s %s\n' "$usuario" "$shell"
> done < /etc/passwd
kali         /usr/bin/bash
```

`break n` e `continue n` **saem de *n* níveis** — `break 2` abandona os dois laços aninhados e dispensa a variável `encontrado=1`. E **redirecione o laço inteiro** (`done > saida.txt`) em vez de `>>` a cada volta: o arquivo é aberto uma vez, não cinco mil.

A VARIÁVEL QUE SOME NO `done`

SHELL PAI
`total=0` continua valendo **0** depois do laço. Nenhum erro é emitido.

SUBSHELL DO PIPE
a cópia de `total` chega a 4 — e morre com o processo.

`cat arq | while read l; do (( total++ )); done` — cada estágio de um *pipe* roda no **próprio processo**. O sintoma é sempre o mesmo: valor certo *dentro* do laço, valor antigo depois do `done`.

- **`done < arq`** redirecionamento: sem *pipe*, sem *fork*. A forma preferida quando a origem é um arquivo.
- **`done < <(cmd)`** substituição de processo — o laço fica no pai. O **espaço entre os dois `<` é obrigatório**.
- **`shopt -s lastpipe`** o último estágio passa a rodar no pai (exige `set +m` em script).

Um erro de lógica num `if` atinge um arquivo; num laço, atinge todos os que casaram. Por isso todo laço destrutivo se escreve **duas vezes**: primeiro com `echo "APAGARIA: $arq"` no lugar da ação, para ler a lista inteira; só depois se troca o `echo` por `rm -f -- "$arq"`. O `--` impede que um arquivo chamado `-rf` seja lido como opção — vale para `rm`, `cp`, `mv` e `chown`. **Nunca rode laço destrutivo com lista vinda de variável não validada:** um `$DIR` vazio transforma `rm -rf "$DIR"/*` em `rm -rf /*`. Ensaie num diretório descartável de `mktemp -d` e use `rm -i` na primeira execução real.

RESUMO RÁPIDO

- ✓ **O `for` itera sobre palavras, não sobre arquivos.** Quem decide as palavras é a expansão que gera a lista: `for f in *.log`, **nunca** `for f in $(ls *.log)`.
- ✓ **Glob sem correspondência passa literalmente** — `shopt -s nullglob` ou `[[ -e $f ]] || continue`. E `**` recursivo exige `shopt -s globstar`, desligado por padrão em toda distribuição.
- ✓ **`for (( i=0; i<n; i++ ))` quando o limite é variável**, porque a expansão de chaves ocorre antes da de variáveis e `{1..$n}` vira texto literal.
- ✓ **`while` / `until` reavaliam a condição a cada volta**: é a ferramenta de esperar por estado externo — sempre com `sleep` e com **limite de tentativas**, senão o script espera para sempre.
- ✓ **O idioma é `while IFS= read -r linha; do … done < arquivo`**, com `|| [[ -n $linha ]]` quando o arquivo puder não terminar em quebra de linha.
- ✓ **`cmd | while read` perde as variáveis no `done`** — o lado direito do *pipe* é subshell. Corrija com `< arquivo` ou `< <(comando)`.
- ✓ **Para `find`, o par seguro é `-print0` + `read -r -d ''`.** Corpo com uma só chamada? Prefira `-exec … +` ou `xargs -0`.

NO KALI

O laço amplifica tanto o acerto quanto o engano — por isso o filtro de escopo vai *dentro* dele, não depois. Valide cada alvo contra a faixa contratada logo na primeira linha do corpo: `[[ $alvo =~ ^10\.10\.10\. ]] || { echo "fora do escopo: $alvo" >&2; continue; }`. Um erro de digitação numa lista de 254 endereços não produz um alvo errado, produz 254 — e varrer fora do escopo autorizado é crime pela Lei 12.737/2012. **`for host in {1..254}` é conveniente e quase sempre a ferramenta errada:** `nmap`, `masscan` e `fping` já varrem em paralelo com controle de temporização, e `nmap -sn 10.0.0.0/24` leva segundos onde o laço serial leva minutos. Use o laço para **orquestrar** — iterar sobre arquivos de saída, perfis de varredura, alvos já confirmados —, não para substituir a ferramenta. **O `sleep` é controle de intensidade, não enfeite:** laço sem pausa contra um serviço de laboratório é indistinguível de uma negação de serviço acidental, e derruba justamente o alvo que você precisava analisar. **Saída de `gobuster` e `nikto` vira nome de arquivo com `%20`, `&`, `?` e espaço** — exatamente a família que quebra `for f in $(ls)`: percorra com glob ou `find -print0` e cite toda expansão. E no *rolling*, cheque a ferramenta **antes** de entrar no laço (`command -v nmap >/dev/null || exit 1`): dentro dele, o teste custa 254 chamadas e 254 mensagens idênticas.